

Task 1 — Instructor Student Progress Dashboard

Tier S (Hardest) | Team 1: Ayman & Daniel | US5

Overview

Teachers currently have no consolidated view of how students are progressing through peer reviews. This task builds a dedicated Student Progress Dashboard that aggregates group membership, review submission counts, and rubric completion across every assignment in a course. The result is a filterable, sortable table that gives instructors an at-a-glance picture of participation and completion.

This is Tier S because it requires multiple complex backend aggregations joining six tables (User, Course, User_Course, Group_Members, Review, Criterion) and a rich interactive frontend page with filtering, sorting, and progress indicators.

Item	Detail
Route prefix	/teacher/classes/<course_id>/progress
New backend file	teacher_controller.py (new endpoint added)
New frontend page	StudentProgressPage.tsx + StudentProgressPage.css
New API function	getCourseProgress(courseId) in api.ts
New App.tsx route	/assignments/:courseId/progress

Accessible by	Teachers only (jwt_teacher_required)
----------------------	--------------------------------------

Why This Feature?

US5 requires that instructors can see per-student submission status, per-assignment submission status, and per-student review completion status — all in one place. Right now a teacher must mentally cross-reference the analytics page, the review list, and the group page across multiple assignments to build this picture. This feature surfaces all of it in a single table.

Backend (Ayman Leads, Daniel Navigates)

Step 1 — Understand Existing Models

Before writing any code, read the following models to understand their relationships:

- User_Course — links users to courses (courseID, userID)
- Group_Members — links users to assignment groups (userID, groupID, assignmentID)
- Review — peer reviews (reviewerID, revieweeID, assignmentID)
- Criterion — individual rubric scores per review (reviewID, grade)
- Assignment — assignments per course (courseID, id, name)

NOTE: *DO NOT modify any existing models. This feature is 100% read-only on the backend.*

Step 2 — Add the Progress Endpoint

Add the following endpoint to `api/controllers/teacher_controller.py` after the existing analytics endpoint:

```
@teacher_bp.route('/classes/<int:course_id>/progress', methods=['GET'])
@jwt_teacher_required
def course_student_progress(course_id):
```

The endpoint must return a JSON object with this shape:

```
{
  'course_id': int,
  'course_name': str,
  'assignments': [{ 'id': int, 'name': str }],
  'students': [
    {
      'user_id': int,
      'name': str,
      'email': str,
      'per_assignment': {
        '<assignment_id>': {
          'in_group': bool,
          'reviews_given': int,
          'reviews_received': int,
          'avg_score': float | null
        }
      }
    }
  ]
}
```

Step 3 — Logic Details

For each enrolled student, for each assignment in the course:

- `in_group`: check if a `Group_Members` row exists for (`userID`, `assignmentID`)
- `reviews_given`: count `Review` rows where `reviewerID` = `student.id` AND `assignmentID` matches
- `reviews_received`: count `Review` rows where `revieweeID` = `student.id` AND `assignmentID` matches
- `avg_score`: average of all `Criterion.grade` values across reviews received for this assignment (null if none)

NOTE: Use `db.session.query()` with filters rather than loading entire tables into memory. The course may have many students.

Step 4 — Write the Test

Create `tests/test_student_progress.py`. Required test cases:

- Teacher receives 200 with correct student and assignment data
- Student user receives 403
- Non-existent `course_id` returns 404
- Student with no groups shows `in_group=False` for all assignments

- Review counts and avg_score are accurate after seeding reviews

Frontend (Daniel Leads, Ayman Navigates)

Step 1 — Add the API Function

Add to frontend/src/util/api.ts:

```
export const getCourseProgress = (courseId: number) =>
  fetch(`${BASE_URL}/teacher/classes/${courseId}/progress`, {
    credentials: 'include',
  }).then(res => { maybeHandleExpire(res); return res; });
```

Step 2 — Add Types to types.d.ts

```
interface ProgressPerAssignment {
  in_group: boolean;
  reviews_given: number;
  reviews_received: number;
  avg_score: number | null;
}

interface ProgressStudent {
  user_id: number;
  name: string;
  email: string;
  per_assignment: Record<string, ProgressPerAssignment>;
}

interface CourseProgressResponse {
  course_id: number;
  course_name: string;
  assignments: { id: number; name: string }[];
  students: ProgressStudent[];
}
```

Step 3 — Build the Page

Create frontend/src/pages/StudentProgressPage.tsx. The page must:

- Accept courseId from useParams()

- Fetch from `getCourseProgress(courseId)` on mount
- Render a search input that filters the student rows by name or email
- Render a dynamic table: first column is student name/email, then one column per assignment
- Each cell shows: a green checkmark if `in_group=true`, reviews given/received counts, and `avg_score` (or `—` if null)
- A completion badge shows overall % of students who have given at least 1 review
- Use `StatusMessage` for loading/error states

NOTE: The table is dynamic — the number of columns changes based on how many assignments the course has. Use `assignments[]` array from the API response to build column headers.

Step 4 — Register the Route

In `App.tsx` add:

```
import StudentProgressPage from './pages/StudentProgressPage';

<Route path='/classes/:courseId/progress'
  element={<ProtectedRoute><StudentProgressPage /></ProtectedRoute>} />
```

Step 5 — Link from ClassHome

In `ClassHome.tsx`, add a button visible to teachers only that navigates to the progress page:

```
// Inside the ClassHeaderRight div, after the CSV button
{isTeacher() && (
  <Button onClick={() => navigate(`/classes/${id}/progress`)}>
    Student Progress
  </Button>
)}
```

Definition of Done

✓	Definition of Done — All items must be checked before PR is raised
[]	GET <code>/teacher/classes/<course_id>/progress</code> returns 200 with correctly shaped JSON for a teacher

[]	Student and unauthenticated requests receive 403
[]	Non-existent course returns 404
[]	All 5 backend tests pass (pytest tests/test_student_progress.py -v)
[]	StudentProgressPage.tsx renders a table with one row per enrolled student
[]	Table has one column per assignment in the course
[]	Each cell shows in_group status, reviews given, reviews received, avg_score
[]	Search input filters student rows in real-time without re-fetching
[]	Overall completion badge shows correct percentage
[]	Page shows a loading state while data is fetching
[]	Page shows a clear error message if the fetch fails
[]	'Student Progress' button appears for teachers on ClassHome and navigates correctly
[]	Route /classes/:courseId/progress is registered in App.tsx behind ProtectedRoute